

OpenFrame OSC CTGユーザーガイド

OpenFrame/OSC 7.1



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame OSC CTGユーザーガイド

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame/OSC 7.1

ガイドバージョン: v3.1.2

Website

<http://www.tmaxsoft.com>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®およびTmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの\${INSTALL_PATH}/license/oss_licensesに記載されている事項を参照してください。

目次

このガイドについて	vii
第1章 OpenFrame CTGの紹介	1
1.1. 概要	1
1.2. OpenFrame CTGの動作プロセス	1
1.3. リソース・アダプターの構成	2
1.4. OSCサーバーのTmax環境設定	4
第2章 CTGクラス	7
2.1. 概要	7
2.2. CCILocalTransaction	8
2.3. CICSConnection	9
2.4. CICSConnectionFactory	9
2.5. CICSConnectionSpec	9
2.6. CICSInteraction	10
2.7. CICSInteractionSpec	11
2.8. CICSManagedConnectionFactory	12
2.9. CICSResourceException	13
2.10. ECIconnection	14
2.11. ECIconnectionFactory	15
2.12. ECIconnectionSpec	15
2.13. ECIInteraction	16
2.14. ECIInteractionSpec	18
2.15. ECIManagedConnectionFactory	21
2.16. ECIResourceAdapterRc	21
第3章 使用例	25
3.1. JCAで作成したEJB 2.0の例	25
3.2. COMMAREAバッファ・クラス	29
3.3. サブレット・プログラム	30
3.4. リソース・アダプター設定ファイル	32
索引	37

図目次

[図 1.1] OpenFrame CTGのデータ処理プロセス	1
---------------------------------------	---

このガイドについて

対象読者

本書は、Tmax OpenFrame[®] (以下、OpenFrame)のOSCシステムが提供するOpenFrame CTG(CICS Transaction Gateway)を使用するアプリケーション開発者を対象としています。OpenFrame CTGを使用して、外部からOSCアプリケーション・サーバーが管理するプログラムを実行することができます。

前提知識

本書を理解するには、以下の事項についての知識が必要です。

- OpenFrame/OSCシステム
- UNIXシステム
- TmaxSoftのTPモニターTmax
- CICS

制限事項

本書は、OpenFrame/OSCが提供するCTGクラスについて説明するものです。OpenFrameの基盤環境であるUNIX、OpenFrameのエンジンであるTmax、そしてOSCシステムのリホストの対象であるメインフレームCICSについては、該当するガイドを参照してください。

本書の構成

本書は、計3章で構成されています。

各章の主な内容は以下のとおりです。

- 第1章: OpenFrame CTGの紹介

OpenFrame CTGを実行するためにリソース・アダプターを構成する方法および環境設定方法について説明します。

- 第2章: CTGクラス

CTGクラスの使用方法について説明します。

- 第3章: 使用例

OpenFrame CTGの実行例を紹介します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名、ディレクトリ
<Ctrl>+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	Solaris 11 (SunOS 5.11)以上(32bit、64bit)
	Linux x86 2.6以上(32bit、64bit)
	Linux ia64 2.6以上(32bit、64bit)
ハードウェア	5GB以上のハードディスク
	8GB以上のメモリ
データベース	Tibero 6 FS07
コンパイラー	Micro Focus COBOL、NetCOBOL、OpenFrame COBOL
	OpenFrame PL/I
	OpenFrame ASM
OpenFrame製品群	OpenFrame/Base 7.1

参考

IBMまたはHP-UXプラットフォームをご使用の場合は、TmaxSoftの技術サポート・チームにお問い合わせください。

関連文書

ガイド	説明
OpenFrame OSCインストールガイド	OpenFrame製品のインストールと環境設定について説明しています。
OpenFrame HiDBガイド	データを構造的に格納および管理するHiDBについて説明しています。
OpenFrame TSAMガイド	VSAMに対応されるTSAMについての紹介、およびデータセットの種類、管理方法、処理方法について説明しています。
OpenFrame OSC管理者ガイド	OSCシステムを運用および管理する方法について説明しています。
OpenFrame OSCリソース定義ガイド	OSCシステムを使用するために必要なリソース定義について説明しています。
OpenFrame OSCマッピングサポートガイド	OSCプログラムとTN3270端末が通信するためのプログラミング・インターフェースを提供するマッピング・サポートについて説明しています。
OpenFrame ユーティリティリファレンスガイド	OpenFrameが提供するさまざまなユーティリティ・プログラムについて説明しています。
OpenFrame エラーメッセージリファレンスガイド	製品の使用時に発生し得るエラーに関する情報と、その対応方法について説明しています。
OpenFrame ツールリファレンスガイド	OpenFrameシステムの運用時に使用するツール・プログラムについて説明しています。
OpenFrame マイグレーションガイド	メインフレーム環境のリソースをOpenFrame環境にマイグレーションする方法について説明しています。

参考文献

製品	ガイド
IBM Mainframe	CICS Application Programming Guide
	CICS Application Programming Reference
	CICS System Programming Reference
	CICS Resource Definition Guide
	CICS Intercommunication Guide

第1章 OpenFrame CTGの紹介

本章では、OpenFrame CTGを実行するためにリソース・アダプターを構成する方法および環境設定方法について説明します。

1.1. 概要

OpenFrame/OSCは、OSCアプリケーション・サーバーが管理するプログラムを外部から実行できるOpenFrame CTG(CICS Transaction Gateway)プログラムを提供します。

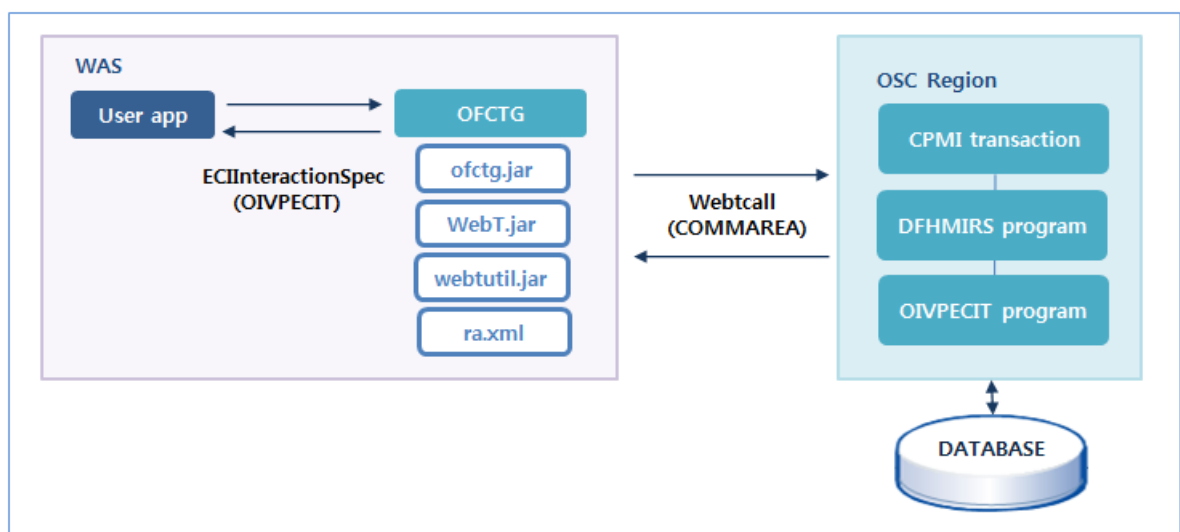
OpenFrame CTGは、OpenFrameのトランザクションを呼び出すためのAPIであるECI(External Call Interface)を提供します。OpenFrameがサポートしていない外部アプリケーション・プログラムは、ECIを利用してOpenFrame OSCリージョンのリソースとして登録されたプログラム(COBOL、PL/I)を呼び出します。

本章では、JCA準拠のOpenFrame CTGの動作プロセスと環境設定について説明します。OpenFrame CTGは、J2EE Connector Architecture(J2CA)仕様に基づいたリソース・アダプターとして機能するライブラリ・パッケージです。

1.2. OpenFrame CTGの動作プロセス

以下の図は、OpenFrame CTGを使用してアプリケーションとOSCリージョン間のデータを処理するプロセスを示しています。

[図 1.1] OpenFrame CTGのデータ処理プロセス



1. WAS(Web Application Server)上で実行されるJavaアプリケーション・プログラムが、ECIインターフェースを介してOIVPECITプログラムの実行を要求します。
2. OFCTG(OpenFrame CTG)を使用するために、Webアプリケーション・サーバーのベンダー固有の設定とJCA設定を行います。詳細については、「[1.3. リソース・アダプターの構成](#)」を参照してください。
3. 要求を受けたOSCリージョンは、CPMIデフォルト・トランザクションを実行します。
4. トランザクションに該当するDFHMIRSプログラムが実行されます。
5. DFHMIRSプログラムは、要求されたOIVPECITプログラムをリンクを使用して呼び出し、COMMAREAを渡します。
6. OIVPECITプログラムは、データベースにアクセスして必要なデータを取得します。
7. データをCOMMAREAに保存して返すと、クライアントにCOMMAREAのデータと戻り値が返されます。

1.3. リソース・アダプターの構成

OpenFrame CTGを使用するには、まずリソース・アダプターを構成する必要があります。リソース・アダプターには、Webアプリケーション・サーバーのベンダー固有の設定とJCA設定を行います。

com.tmax.ofctg.connector.cicsパッケージ・クラスを利用する場合は、ra.xmlファイルにリソース・アダプターに設定可能な名前とタイプを設定する必要があります。ra.xmlの設定例は「[3.4. リソース・アダプター設定ファイル](#)」を参照してください。

以下は、リソース・アダプターの設定項目についての説明です。

プロパティ名	プロパティ・タイプ	プロパティ値
backup	java.lang.String	OSCサーバーのバックアップ・ホスト・アドレスを指定します。
backup_port	java.lang.Integer	OSCサーバーのバックアップ・ホストのポート番号を指定します。support_xaをtrueに指定した場合は、OSCのTmax環境設定に定義されたJEUSGWポート番号を指定する必要があります。
connectionName	java.lang.String	ロギングなどの際に内部で使用する接続名を指定します。
connectionTimeout	java.lang.Integer	OSCサーバーへの接続タイムアウトを指定します。 (単位: 秒)
encryption	java.lang.Boolean	暗号化するかどうかを指定します。 TmaxPortで接続する場合にのみ使用できます。この場合は、support_xaをfalseに指定してください。XAを使

プロパティ名	プロパティ・タイプ	プロパティ値
		用するためにJEUSGWと通信する場合には使用できません。
headerType	java.lang.String	OpenFrame CTGがOSCサーバーと通信時に使用するプロトコルを指定します。現在は、extendedV4のみサポートしています。
host	java.lang.String	OSCサーバーのホスト・アドレスを指定します。
host_port	java.lang.Integer	OSCサーバーのホストのポート番号を指定します。 support_xaをtrueに指定した場合は、OSCのTmax環境設定に定義されたJEUSGWポート番号を指定する必要があります。
logBufSize	java.lang.Integer	OpenFrame CTGがログを記録時に使用するバッファのサイズです。ログ・サイズが設定値に達すると、ファイルに記録されます(デフォルト値: 512バイト)。
logDir	java.lang.String	OpenFrame CTGが作成するログ・ファイルを保存するパスを指定します。
logFile	java.lang.String	OpenFrame CTGが作成するログ・ファイルの名前を指定します。
logFileFormat	java.lang.String	OpenFrame CTGが作成するログ・ファイル名の後ろに付ける日付の形式を指定します(デフォルト値: MMddyyyy)。
logLevel	java.lang.String	OpenFrame CTGがログを記録時に使用するログ・レベルを指定します。 – info (デフォルト値) – debug : 内部動作をトレースするためのログを出力します。 – dev : 送信データを含むさまざまなログを記録します。
logValidDays	java.lang.Integer	1を指定します。
regionName	java.lang.String	接続するリージョン名を指定します。
support_xa	java.lang.Boolean	XAをサポートするかどうかを指定します。trueまたはfalseのいずれかを指定します。
tpTimeout	java.lang.Integer	要求に対する応答待ち時間を指定します。 (未指定時のデフォルト値: 60秒)
txHelperClassName	java.lang.String	TransactionManagerを返すトランザクション・ヘルパー・クラス名を指定します。

プロパティ名	プロパティ・タイプ	プロパティ値
		XAをサポートする場合は、getConnection()の呼び出し時に指定された名前を持つクラスのTransactionManagerにXAResourceを登録(enlist)します。現在は、指定したヘルパー・クラスがgetTransactionManager()を呼び出します。
xidMapperFile	java.lang.String	OpenFrame CTGが使用するXidMapperFileの名前を指定します。 xidMapperFileは、WASのTransactionManagerが提供するXIDと、OSCで使用するXIDのマッピングおよび決定情報を記録するために使用されます。
xidMapperPath	java.lang.String	XidMapperFileのパスを指定します。

1.4. OSCサーバーのTmax環境設定

JCAを使用してグローバル・トランザクションを使用するには、OSCサーバーのTmax環境設定ファイルに以下の設定を行う必要があります。

```
*GATEWAY
JEUSGW      GWTYPE = JEUS, PORTNO = 9000,
             DIRECTION=IN,
             NODENAME = NODE1,
             CLOPT = "-r -h 4 -D 7 -o $(SVR)$(CDATE).out -e $(SVR)$(CDATE).err",
             CPC = 2
```

項目	説明
GWTYPE	必ず「JEUS」を指定します。WASをリモート・システムとして指定します。Oracle WebLogic、IBM WebSphereなど、JEUS以外のWASで使用する場合にも「JEUS」を指定してください。
PORTNO	ゲートウェイが使用するリッスン・ポート番号を指定します。
DIRECTION	ゲートウェイとWAS間の要求の方向を指定します。 「IN」を指定すると、受信する要求のみ処理できます。
NODENAME	ゲートウェイが実行されるノード名を指定します。Tmax環境設定のNODEセクションに登録された名前を指定します。
MAXINRGW	ゲートウェイに接続できるチャネルの最大数を指定します（デフォルト値:32、最大値:128）。
CLOPT	ゲートウェイが使用するオプションを指定します。 CLOPT = "-r -h <n> -D <n> -o <filename> -e <filename>"

項目	説明
	-r : OpenFrame CTGとJEUSGW間のトランザクション・リカバリ機能を使用することを指定します。 -h <n> : OpenFrame CTGとJEUSGW間の通信プロトコルを指定します。必ず「-h 4」を指定します。リソース・アダプターの設定では、headerType項目をextendedV4に指定します。 -D <n> : ゲートウェイのログ・レベルを指定します。 -o <filename> : ゲートウェイの標準出力ログを記録するファイルを指定します。 -e <filename> : ゲートウェイの標準エラー出力ログを記録するファイルを指定します。
CPC	ゲートウェイとCLH間の並列通信チャネルの数を指定します。 処理量が多くサービスの処理に時間がかかる場合は、複数の通信チャネルを使用して並列で処理することによって、処理速度を向上させます。

参考

Tmax環境設定ファイルについての詳細は、『Tmax 管理者ガイド』の「3.2 基本環境設定」を参照してください。

第2章 CTGクラス

本章では、CTGクラスの使用方法について説明します。

2.1. 概要

以下は、J2EE ECIアプリケーション・プログラムの開発のために提供するクラスの一覧と、発生する可能性のある例外クラスの一覧です。続く節では、各クラスについて説明します。

- 開発クラス

クラス	説明
CCILocalTransaction	ECIコネクタのトランザクション状態を制御します。
CICSConnection	現在、単独では使用できません。（ ECIConnection を参照）
CICSConnectionFactory	現在、単独では使用できません。（ ECIConnectionFactory を参照）
CICSConnectionSpec	現在、単独では使用できません。（ ECIConnectionSpec を参照）
CICSInteraction	現在、単独では使用できません。（ ECIInteraction を参照）
CICSInteractionSpec	現在、単独では使用できません。（ ECIInteractionSpec を参照）
CICSManagedConnectionFactory	現在、単独では使用できません。（ ECIManagedConnectionFactory を参照）
ECIConnection	OSCサーバーへの接続を表すクラスです。このオブジェクトを介して対話(Interaction)を作成し、CICSに送受信します。
ECIConnectionFactory	OSCサーバーへの接続を取得するクラスです。
ECIConnectionSpec	OSCサーバーへの接続のプロパティを定義したクラスです。getConnectionを実行するとき、このクラスで定義されたプロパティが使用されます。
ECIInteraction	ECIConnectionオブジェクトを介したOSCサーバーとの対話を表すクラスです。
ECIInteractionSpec	CICSサーバーとの対話のプロパティを定義したクラスです。
ECIManagedConnectionFactory	ECIConnectionオブジェクトを作成するために使用される上位クラスです。このクラスには、CICSサーバーへの接続に必要なすべての情報が含まれています。

- 例外クラス

例外	説明
ECIResourceAdapterRc	CICSResourceExceptionクラスのエラーコード定数を定義したクラスです。OSCアプリケーション・サーバーの実行中に発生したエラーの定数を定義しています。
CICSResourceException	ECIInteractionで要求を処理中に発生したエラーに関する例外情報を定義したクラスです。

2.2. CCILocalTransaction

CCILocalTransactionクラスは、ECIコネクタのトランザクション状態を制御します。
ECIConnection.getLocalTransaction()メソッドによってのみアクセスできます。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class CCILocalTransaction
implements javax.resource.cci.LocalTransaction
```

メソッド

以下は、CCILocalTransactionクラスのメソッドです。

メソッド	説明
public void begin()	新しいローカル・トランザクションを開始します。 ● スローする例外 – RuntimeException : トランザクションの開始に失敗した場合に発生します。
public void commit()	現行のローカル・トランザクションをコミットします。 ● スローする例外 – RuntimeException : トランザクションのコミットに失敗した場合に発生します。
public void rollback()	現行のローカル・トランザクションをロールバックします。 ● スローする例外

メソッド	説明
	– <code>ResourceException</code> : トランザクションのロールバックに失敗した場合に発生します。

2.3. CICSCConnection

現在、CICSCConnectionを単独では使用できません。詳細については、[ECIConnection](#)を参照してください。

2.4. CICSCConnectionFactory

現在、CICSCConnectionFactoryを単独では使用できません。詳細については、[ECIConnectionFactory](#)を参照してください。

2.5. CICSCConnectionSpec

現在、CICSCConnectionSpecを単独では使用できません。詳細については、[ECIConnectionSpec](#)を参照してください。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class CICSCConnectionSpec
implements javax.resource.cci.ConnectionSpec
```

メソッド

以下は、CICSCConnectionSpecクラスのメソッドです。

メソッド	説明
<pre>public void setUserName(java.lang.String newUser)</pre>	プログラムの実行時にTACF認証に使用されるユーザー名を設定します。ECIInteractionSpecクラスのsetUserID()が呼び出された場合は、このメソッドを使用して設定した値は無視されます。
<pre>public java.lang.String getUserName()</pre>	プログラムの実行時にTACF認証に使用されるユーザー名を返します。

メソッド	説明
public void setPassword(java.lang.String newPass)	プログラムの実行時にTACF認証に使用されるパスワードを設定します。ECIInteractionSpecクラスのsetPassword()が呼び出された場合は、このメソッドを使用して設定した値は無視されます。
public java.lang.String getPassword()	プログラムの実行時にTACF認証に使用されるパスワードを返します。
String getHost()	OSCサーバーのホストのアドレスを取得します。
void setHost(String host)	OSCサーバーのホストのアドレスを設定します。
int getHost_port()	OSCサーバーのホストのポート番号を取得します。
void setHost_port(Integer host_port)	OSCサーバーのホストのポート番号を取得します。
String getBackup()	OSCサーバーのバックアップ・ホストのアドレスを取得します。
void setBackup(String backup)	OSCサーバーのバックアップ・ホストのアドレスを設定します。
int getBackup_port()	OSCサーバーのバックアップ・ホストのポート番号を取得します。
void setBackup_port(Integer backup_port)	OSCサーバーのバックアップ・ホストのポート番号を設定します。
int getConnectTimeout()	OSCサーバーへの接続タイムアウトを取得します(単位: 秒)。
void setConnectTimeout(Integer connectTimeout)	OSCサーバーへの接続タイムアウトを設定します。

2.6. CICSInteraction

CICSサーバーとの対話を表すクラスです。このクラスは接続情報を保持し、CICSサーバーとの送受信を実行します。現在、CICSInteractionを単独では使用できません。詳細については、[ECIInteraction](#)を参照してください。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class CICSInteractionSpec  
implements javax.resource.cci.Interaction
```

メソッド

以下は、CICSInteractionクラスのメソッドです。

メソッド	説明
public Connection getConnection()	自身のインスタンス(CICSInteraction)を作成したECIConnectinonクラスのインスタンスを返します。
public ResourceWarning getWarnings()	対話でexecute()メソッドの実行中にエラーが発生した場合、エラー情報が含まれているCICSResourceExceptionクラスを返します。 エラーが発生していない場合は、NULLを返します。 - スローする例外 - ResourceException : ResourceWarning情報のリターンに失敗した場合に発生します。
public void clearWarnings()	対話中に保存されたResourceWarning情報を削除します。 - スローする例外 - ResourceException : ResourceWarning情報の削除に失敗した場合に発生します。
public boolean execute(InteractionSpec aspec, Record sndRecord, Record rcvRecord)	このメソッドは直接呼び出すことができません。必ずECIInteractionクラスを使用します。 - スローする例外 - ResourceException : CICSサーバーとのメッセージの送受信に失敗した場合に発生します。
public void close()	対話に該当するECIConnection接続を閉じます。 - スローする例外 - ResourceException : 接続のクローズに失敗した場合に発生します。

2.7. CICSInteractionSpec

現在、CICSInteractionSpecを単独では使用できません。詳細については、[ECIInteractionSpec](#)を参照してください。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class CICSInteractionSpec  
implements javax.resource.cci.InteractionSpec
```

メソッド

以下は、CICSInteractionSpecクラスのメソッドです。

メソッド	説明
public int getInteractionVerb()	当該インスタンスに保存されたInteractionVerbを取得します。使用される対話のタイプを示します。 以下のいずれかの値を返します。 – SYNC_SEND : 現在サポートしていません。 – SYNC_RECEIVE : 現在サポートしていません。 – SYNC_SEND_RECEIVE : DPL方式で要求を送信し、応答を受信します。
public void setInteractionVerb(int newVerb)	新しいInteractionVerbを設定します。 – newVerb : SYNC_SEND、SYNC_RECEIVE、SYNC_SEND_RECEIVEのいずれかの値を指定します。現在は、SYNC_SEND_RECEIVEのみサポートしています。
public void addPropertyChangeListener(PropertyChangeListener pcl)	PropertyChangeListenerを追加します。
public void removePropertyChangeListener(PropertyChangeListener pcl)	PropertyChangeListenerを削除します。

2.8. CICSManagedConnectionFactory

現在、CICSManagedConnectionFactoryを単独では使用できません。詳細については、[ECIManagedConnectionFactory](#)を参照してください。各メソッドのデフォルト設定値は、リソース・アダプターに定義した設定値が保存されています。

- パッケージ


```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class CICSManagedConnectionFactory
implements com.tmax.ofctg.connector.cics.CICSLogTraceLevels,
    java.io.Serializable,
    javax.resource.spi.ManagedConnectionFactory,
    javax.resource.spi.ResourceAdapterAssociation,
    javax.resource.spi.ValidatingManagedConnectionFactory
```

2.9. CICSResourceException

ECIInteractionで要求を処理中に発生したエラーに関する例外情報を保持しています。

ECIInteractionクラスのgetWarnings()メソッドを使用して最初の例外情報を取得することができます。また、ECIInteractionクラスのexecute()メソッドを実行中に例外が発生した場合は、ABENDコードなどの追加エラー情報を取得することができます。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class CICSResourceException
extends javax.resource.cci.ResourceWarning
```

メソッド

以下は、CICSResourceExceptionクラスのメソッドです。

メソッド	説明
public String getAbendCode()	要求を処理中に異常終了が発生した場合に、ABENDコードを返します。ABENDコードが存在しない場合は、NULLを返します。
public int getCicsRespCode()	ECIResourceAdapterRcクラスに定義されている応答コード定数を返します。エラーの詳細については、 ECIResourceAdapterRc を参照してください。
public int getCicsServerStatus()	ECIResourceAdapterRcクラスに定義されているサーバー・ステータス定数を返します。エラーの詳細については、 ECIResourceAdapterRc を参照してください。

2.10. ECIConnection

OSCサーバーへの接続を表すクラスです。このオブジェクトを介して対話(Interaction)を作成し、CICSに送受信します。ECIConnectionFactoryクラスのgetConnection()メソッドを使用してオブジェクトを作成します。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class ECIConnection
extends CICSConnection
implements javax.resource.cci.Connection
```

メソッド

以下は、ECIConnectionクラスのメソッドです。

メソッド	説明
public void close()	接続を閉じます。 ● スローする例外 – RuntimeException : 接続のクローズに失敗した場合に発生します。
public Interaction createInteraction()	接続を介して実行するECIInteractionを作成します。 ● スローする例外 – RuntimeException : ECIInteractionの作成に失敗した場合に発生します。
public LocalTransaction getLocalTransaction()	接続のトランザクション状態を制御するために使用されるローカル・トランザクション・オブジェクトを取得します。 ● スローする例外 – RuntimeException : ローカル・トランザクションの作成に失敗した場合に発生します。

2.11. ECIConnectionFactory

ECI接続を取得するクラスです。ユーザーはリソース・アダプターの設定によりこのクラスをルックアップします。getConnectionを使用してリソース・アダプターの設定、または接続仕様の設定が適用された接続を取得します。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class ECIConnectionFactory  
extends com.tmax.ofctg.connector.cics.CICSConnectionFactory
```

メソッド

以下は、ECIConnectionFactoryクラスのメソッドです。

メソッド	説明
public Connection getConnection(ConnectionSpec connectionSpec)	接続のトランザクション状態を制御するために使用されるローカル・トランザクション・オブジェクトを取得します。 ● パラメータ – connectionSpec : 接続に適用する仕様です。 接続を作成するときは、デフォルトでファクトリに設定されたプロパティ値が適用されます。このパラメータがNULLではなく、host、host_port、backup、backup_port、connectionTimeoutが設定された場合は、接続仕様に設定された値が優先して適用されます。ECIConnectionSpecオブジェクトを作成して、引数として渡します。 ● スローする例外 – <code>ResourceException</code> : 接続の作成に失敗した場合に発生します。

2.12. ECIConnectionSpec

ECI接続のプロパティを定義したクラスです。getConnectionの実行時に、このクラスのオブジェクトが保持する接続プロパティを使用します。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class ECISpecConnectionSpec
extends com.tmax.ofctg.connector.cics.CISConnectionSpec
```

コンストラクター

メソッド	説明
public ECISpecConnectionSpec()	ECISpecConnectionSpecオブジェクトを作成します。
public ECISpecConnectionSpec(java.lang.String user, java.lang.String pass)	ユーザー名とパスワードを使用して新しい ECISpecConnectionSpecを作成します。

メソッド

詳細については、[CISConnectionSpec](#)を参照してください。

2.13. ECInteraction

EC接続を介したOSCリージョン・サーバーとの対話を表すクラスです。このクラスは接続情報を保持し、その情報に基づいてOSCリージョン・サーバーとの送受信を実行します。ECISpecConnectionクラスのcreateInteraction()メソッドを使用してオブジェクトを作成します。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class ECInteraction
extends com.tmax.ofctg.connector.cics.CISInteraction
```

メソッド

以下は、ECInteractionクラスのメソッドです。

メソッド	説明
public Connection getConnection()	自身のインスタンス(CICSConnection)を作成したECIConnectionクラスのインスタンスを返します。
public ResourceWarning getWarnings()	対話でexecute()メソッドの実行中にエラーが発生した場合、エラー情報が含まれているCICSResourceExceptionを返します。 エラーが発生していない場合は、NULLを返します。 - スローする例外 - ResourceException : ResourceWarning情報のリターンに失敗した場合に発生します。
public void close()	対話に該当するECIConnection接続を閉じます。 - スローする例外 - ResourceException : 接続のクローズに失敗した場合に発生します。
public boolean execute(InteractionSpec aspect, Record sndRecord, Record rcvRecord)	- パラメータ - aspect : 実行する対話に該当するInteractionSpecです。 - sndRecord : 送信するデータが含まれているレコード・オブジェクトです。 - rcvRecord : 受信するデータが含まれているレコード・オブジェクトです。 [注意] レコード・インターフェースはSerializableインターフェースを継承しますが、Javaオブジェクトのデータを非JVM環境に送信する場合には不適切です。したがって、レコード・インターフェースとjavax.resource.cci.Streamableインターフェースの両方を実装して、レコード・インスタンスのデータをCOMMAREA形式に合わせて送信するように作成する必要があります。現在、execute()メソッドでは、レコード・オブジェクトにStreamableが含まれていない場合、例外としてCICSResourceExceptionを発生させます。 - 戻り値 - true : 正常に実行された場合です。 - false : 実行に失敗した場合です。 - スローする例外

メソッド	説明
	– CICSResourceException : 実行に失敗した場合に発生します。エラー状況に応じて、CICSのABENDコードを確認することができます。 – ResourceException : 実行に失敗した場合に発生します。

2.14. ECIIInteractionSpec

ECI接続とCICSサーバーとの対話のプロパティを定義したクラスです。ECIInteractionは、CICSサーバーとの送受信を実行するときに、ECIIInteractionSpecに定義されたプロパティを使用します。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class ECIIInteractionSpec
implements java.io.Serializable,
    javax.resource.cci.InteractionSpec
```

コンストラクター

コンストラクター	説明
public ECIIInteractionSpec()	ECIIInteractionSpecクラスのオブジェクトを作成します。 次の値を設定します。 – InteractionVerb : SYNC_SEND_RECEIVE – UserID, Password : 使用しません。(setUserID()、setPassword()を使用して変更可能) – Commarea Length : 5000バイト (setCommareaLength()を使用して変更可能)
public ECIIInteractionSpec(int verb, String prog)	ECIIInteractionSpecクラスのオブジェクトを作成します。デフォルト値については、public ECIIInteractionSpec()コンストラクターを参照してください。 ● パラメータ – verb : InteractionVerbを指定します。

コンストラクター	説明
	SYNC_SEND、SYNC_RECEIVE、SYNC_SEND_RECEIVEのいずれかを指定します。（現在は、SYNC_SEND_RECEIVEのみサポートしています） – prog : 実行するプログラム名を指定します。 setFunctionName()メソッドを使用して変更できます。
public ECInteractionSpec(int verb, String prog, int timeout, int commLen, int repLen)	ECInteractionSpecクラスのオブジェクトを作成します。デフォルト値については、public ECInteractionSpec()コンストラクターを参照してください。 ● パラメータ – verb : InteractionVerbを指定します。 SYNC_SEND、SYNC_RECEIVE、SYNC_SEND_RECEIVEのいずれかを指定します（現在は、SYNC_SEND_RECEIVEのみサポートしています）。 – prog : 実行するプログラム名を指定します。 setFunctionName()メソッドを使用して変更できます。 – timeout : 応答タイムアウトを指定します。（現在はサポートしていません） – commLen : COMMAREAの長さを指定します。 – repLen : 応答データの長さを指定します。現在は、この設定値と関係なく、すべての応答データを受信します。

メソッド

以下は、ECInteractionSpecクラスのメソッドです。

メソッド	説明
public int getCommareaLength()	COMMAREAの長さを返します。
public int getExecuteTimeout()	ECInteractionSpecクラスのexecute()メソッドを実行時のタイムアウトを取得します。
public int getInteractionVerb()	InteractionVerbを返します。 InteractionSpecクラスに定義されているSYNC_SEND、SYNC_RECEIVE、SYNC_SEND_RECEIVEのいずれかを返します。

メソッド	説明
<code>public String getFunctionName()</code>	プログラム名を返します。
<code>public String getPassword()</code>	プログラムの実行時にTACF認証に使用されるパスワードを返します。
<code>public int getReplyLength()</code>	応答データの長さを返します。現在は、この設定値と関係なく、すべての応答データを受信します。
<code>public String getTPNName()</code>	TP名を返します。
<code>public String getTranName()</code>	Tran名を返します。
<code>public String getUserID()</code>	プログラムの実行時にTACF認証に使用されるIDを返します。
<code>public void setCommareaLength(int length)</code>	COMMAREAの長さを指定します。
<code>public void setExecuteTimeout(int time)</code>	ECIInteractionクラスのexecute()メソッドを実行時のタイムアウトを指定します。このメソッドを呼び出さない場合は、デフォルト値としてリソース・アダプターの設定値を使用します。
<code>public void setFunctionName(String name)</code>	呼び出すプログラム名を指定します。
<code>public void setInteractionVerb(int verb)</code>	InteractionVerbを指定します（現在は、SYNC_SEND_RECEIVEのみサポートしています）。
<code>public void setUserID(String password)</code>	プログラムの実行時にTACF認証に使用されるIDを指定します。 この値が設定された場合は、 ECIConnectionSpec クラスのsetUserName()メソッドより優先して適用されます。
<code>public void setPassword(String password)</code>	プログラムの実行時にTACF認証に使用されるパスワードを指定します。 この値が設定された場合は、 ECIConnectionSpec クラスのsetPassword()メソッドより優先して適用されます。
<code>public void setReplyLength(int length)</code>	応答データの長さを指定します。現在は、この設定値と関係なく、すべての応答データを受信します。
<code>public void setTPNName(String newTPNName)</code>	ECI呼び出しによって実行されるミラー・トランザクション名を指定します。 TPNNameとTranNameがいずれも指定されていない場合は、デフォルト値のCPMIがミラー・トランザクション名として使用されます。ただし、TPNNameとTranNameの両方を指定した場合は、TPNNameが優先して適用されます。
<code>public void setTranName(java.lang.String newTranName)</code>	ECI呼び出しによって実行されるミラー・トランザクション名を指定します。

メソッド	説明
	TPNNameとTranNameがいずれも指定されていない場合は、デフォルト値のCPMIがミラー・トランザクション名として使用されます。ただし、TPNNameを指定せずにTranNameのみ指定した場合は、ミラー・トランザクションにはデフォルト値のCPMIが適用され、OSCのEIBTRNIDにはTranNameが適用されます。

2.15. ECIManagedConnectionFactory

ECIManagedConnectionFactoryは、ECI接続を作成するために使用される上位クラスです。このクラスには、CICSサーバーに接続するためのすべての情報が含まれています。該当の情報は、リソース・アダプターのra.xmlに設定します。ra.xmlの設定例については、[「3.4. リソース・アダプター設定ファイル」](#)を参照してください。

ユーザーはこのクラスを直接使用するのではなく、リソース・アダプターに設定された名前でルックアップを実行すると、createConnectionFactoryによって作成されたECIConnectionFactoryを取得して使用することになります。リソース・アダプターに設定できる名前とタイプについては、[CICSManagedConnectionFactory](#)を参照してください。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public final class ECIManagedConnectionFactory
extends com.tmax.ofctg.connector.cics.CICSManagedConnectionFactory
implements javax.resource.spi.ManagedConnectionFactory,
    javax.resource.spi.TransactionSupport
```

2.16. ECIResourceAdapterRc

OSCアプリケーション・サーバーの実行中に発生したエラーの定数を定義したクラスです。

- パッケージ

```
com.tmax.ofctg.connector.cics
```

- クラスの定義

```
public class ECIResourceAdapterRc
```

メソッド

以下は、ECIResourceAdapterRcクラスのメソッドです。

メソッド	説明
public static String getErrorMessage(int cicsRespCode)	エラー定数の文字列を返します。 ● 応答コード – ECI_NO_ERROR – ECI_ERR_INVALID_DATA_LENGTH – ECI_ERR_INVALID_EXTEND_MODE – ECI_ERR_NO_CICS – ECI_ERR_CICS_DIED – ECI_ERR_RESPONSE_TIMEOUT – ECI_ERR_TRANSACTION_ABEND – ECI_ERR_LUW_TOKEN – ECI_ERR_SYSTEM_ERROR – ECI_ERR_THREAD_CREATE_ERROR – ECI_ERR_INVALID_DATA_AREA – ECI_ERR_UNKNOWN_SERVER – ECI_ERR_INVALID_TRANSID – ECI_ERR_SECURITY_ERROR ● サーバーの状態：サーバーの状態を返します。 – ECI_SERVERSTATE_DOWN – ECI_SERVERSTATE_UNKNOWN – ECI_SERVERSTATE_UP 各コードについての詳細は、「ECIResourceAdapterRcクラスの定数」を参照してください。

参考

ECI_ERR_NO_REPLY、ECI_ERR_REQUEST_TIMEOUT、ECI_ERR_EXEC_NOT_RESIDENT、
ECI_ERR_NULL_WIN_HANDLE、ECI_ERR_NULL_MESSAGE_ID、

ECI_ERR_ALREADY_ACTIVE、ECI_ERR_RESOURCE_SHORTAGE、
ECI_ERR_NO_SESSIONS、ECI_ERR_NULL_SEM_HANDLE、ECI_ERR_INVALID_VERSION、
ECI_ERR_MORE_SYSTEMS、ECI_ERR_CALL_FROM_CALLBACK、
ECI_ERR_NO_SYSTEMS、ECI_ERR_MAX_SYSTEMS、ECI_ERR_MAX_SESSIONS、
ECI_ERR_ROLLEDBACK、ECI_ERR_XID_INVALIDなどの応答コードはサポートされません。

ECIResourceAdapterRcクラスの定数

以下は、ECIResourceAdapterRcクラスの定数です。

定数名	値	説明
public static final short ECI_CLIENTSTATE_UNKNOWN	0	クライアントの状態が不明です。
public static final short ECI_CLIENTSTATE_UP	1	クライアントが使用可能です。
public static final int ECI_ERR_CICS_DIED	-4	通信中にリージョン・サーバーが終了されました。
public static final int ECI_ERR_INVALID_CALL_TYPE	-14	ECI_SYNC、ECI_SYNC_TPN以外の値が指定されました。
public static final int ECI_ERR_INVALID_DATA_AREA	-19	COMMAREAの長さ実際に受信したデータの長さが一致しません。
public static final int ECI_ERR_INVALID_DATA_LENGTH	-1	COMMAREAの長さが無効です。
public static final int ECI_ERR_INVALID_EXTEND_MODE	-2	拡張モードがECI_NO_EXTENDではありません。
public static final int ECI_ERR_INVALID_TRANSID	-24	定義されていないトランザクションが呼び出されました。
public static final int ECI_ERR_LUW_TOKEN	-8	不明なLUWトークンです。
public static final int ECI_ERR_NO_CICS	-3	該当のリージョン・サーバーが起動されていません。
public static final int ECI_ERR_RESPONSE_TIMEOUT	-6	応答待ち中にタイムアウトになりました。
public static final int ECI_ERR_SECURITY_ERROR	-27	ユーザーIDとパスワードが無効です。

定数名	値	説明
public static final int ECI_ERR_SYSTEM_ERROR	-9	不明な内部エラーです。
public static final int ECI_ERR_TRANSACTION_ABEND	-7	トランザクションの異常終了が発生しました。
public static final int ECI_ERR_UNKNOWN_SERVER	-22	不明なリージョン・サーバーです。
public static final int ECI_EXTENDED	2	拡張モードがEXTENDEDです。
public static final int ECI_NO_ERROR	0	正常に終了されました。
public static final int ECI_NO_EXTEND	1	拡張モードがNO_EXTENDです。
public static final short ECI_SERVERSTATE_DOWN	2	サーバーが起動されていません。
public static final short ECI_SERVERSTATE_UNKNOWN	0	サーバーの状態が不明です。
public static final short ECI_SERVERSTATE_UP	1	サーバーの状態が正常です。
public static final int ECI_SYNC	1	呼び出しタイプがSYNCに指定されています。
public static final int ECI_SYNC_TPN	2	呼び出しタイプがSYNC_TPNに指定されています。

第3章 使用例

本章では、OpenFrame CTGの実行例を紹介します。

3.1. JCAで作成したEJB 2.0の例

以下は、サブレットがEJBを呼び出して、データベースのテーブルに値を挿入し、OSCサーバーの特定のプログラムを呼び出すプロセスを1つのトランザクションとして処理する例です。

<ejb/server/CtgXa.java>

```
package ejb.server;

import javax.ejb.Remote;

@Remote
public interface CtgXa{
    String doXATest(String data);
}
```

以下は、EJBの実際の動作を実装した例です。

Bean管理トランザクション(BMT)を使用してユーザー・トランザクションを作成して、データベース・テーブルに値を挿入し、OSCリージョン・サーバーに登録されたプログラムを呼び出すプロセスを1つのトランザクションとして処理します。

<ejb/server/CtgXaBean.java>

```
package ejb.server;

import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
import javax.annotation.Resource;
import javax.ejb.SessionContext;
import javax.ejb.Stateless;
import javax.ejb.TransactionManagement;
import javax.ejb.TransactionManagementType;
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.resource.ResourceException;
import javax.resource.cci.ConnectionFactory;
import javax.resource.cci.Interaction;
import javax.resource.cci.InteractionSpec;
import javax.sql.DataSource;
```

```

import javax.transaction.SystemException;
import javax.transaction.UserTransaction;

import com.tmax.ofctg.connector.cics.CICSResourceException;
import com.tmax.ofctg.connector.cics.ECIConnectionSpec;
import com.tmax.ofctg.connector.cics.ECIInteractionSpec;
import com.tmax.ofctg.connector.cics.ECIResourceAdapterRc;

@Stateless(mappedName="ejb.server.DBTest")
@TransactionManagement(TransactionManagementType.BEAN)
public class CtgXaBean implements CtgXa
{
    @Resource
    SessionContext ejbContext;

    public String doXATest(String data) {
        UserTransaction tx = null;
        String dbResult = null;
        String ctgResult = null;

        try {
            tx = this.ejbContext.getUserTransaction();
            tx.begin();

            dbResult = insertToDatabase("INSERT INTO CTG_TEST2 VALUES('TST7')");
            ctgResult = insertToCTG(data);

            tx.commit();
            System.out.println("Transaction Committed.");

        } catch (Exception e) {
            try {
                tx.rollback();
                System.out.println("Rollback Succeeded.");
            } catch (IllegalStateException e1) {
                e1.printStackTrace();
                System.out.println("Rollback failed.");
            } catch (SecurityException e1) {
                e1.printStackTrace();
                System.out.println("Rollback failed.");
            } catch (SystemException e1) {
                e1.printStackTrace();
                System.out.println("Rollback failed.");
            }
        }

        return "DB Result = " + dbResult + " <br> Result2 = " + ctgResult;
    }
}

```

```

public String insertToDatabase(String query) {
    Connection conn = null;
    Statement stmt = null;
    String result = null;

    try {
        Context ctx = new InitialContext();
        DataSource ds = (DataSource) ctx.lookup("tibero_xa_datasource");
        conn = ds.getConnection();

        stmt = conn.createStatement();
        int count = stmt.executeUpdate(query);
        if (count > 0)
            result = String.valueOf(count);

    } catch (SQLException sqle) {
    } catch (Exception e) {
    } finally {
        try {
            if (stmt != null)
                stmt.close();
            if (conn != null)
                conn.close();
        } catch (SQLException e) {
        }
    }
    return result;
}

public String insertToCTG(String commarea) {
    javax.resource.cci.Connection conn = null;
    String returnStr = null;
    String functionName = "PIVPECIT";

    try {
        InitialContext ctx = new InitialContext();
        ConnectionFactory connFactory = (ConnectionFactory)
ctx.lookup("CTGConnector");

        ECISpecification cs = new ECISpecification();
        conn = connFactory.getConnection(cs);
        Interaction action = conn.createInteraction();

        ECIIInteractionSpec eciiSpec = new ECIIInteractionSpec();
        eciiSpec.setUserID("");
        eciiSpec.setPassword("");
    }
}

```

```

        eciiSpec.setFunctionName(functionName);
        eciiSpec.setCommareaLength(5000);
        eciiSpec.setInteractionVerb(InteractionSpec.SYNC_SEND_RECEIVE);

        GenericRecord sndRecord = new GenericRecord(commarea.getBytes("UTF-8"));

        GenericRecord rcvRecord = new GenericRecord();

        if (action.execute(eciiSpec, sndRecord, rcvRecord)) {
            byte[] result = rcvRecord.getCommarea();
            returnStr = new String(result, "UTF-8");
        }
        System.out.println("execute CTG Succeeded.");

    } catch (CICSResourceException e) {
        String abendCode = e.getAbendCode();
        int respCode = e.getCicsRespCode();

        if (respCode != ECIResourceAdapterRc.ECI_NO_ERROR)
            returnStr = "Func: " + functionName + ", Abend: " + abendCode +
                ", Resp: " + ECIResourceAdapterRc.getErrorMessage(respCode);
        else
            returnStr = "Func: " + functionName + ", Abend: " + abendCode;

        System.out.println("execute CTG failed, " + returnStr);

    } catch (ResourceException e) {
        e.printStackTrace();
        return e.getMessage();

    } catch (Exception e) {
        e.printStackTrace();
        return e.getMessage();

    } finally {
        try {
            if (conn != null)
                conn.close();
        } catch (ResourceException e) {
            e.printStackTrace();
        }
    }

    return returnStr;
}
}

```


3.2. COMMAREAバッファ・クラス

OSCサーバーとの通信に使用されるCOMMAREAバッファを表すクラスです。

javax.resource.cci.Streamableインターフェースを実装する必要があります。

<ejb/server/GenericRecord.java>

```
package ejb.server;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;

import javax.resource.cci.Record;
import javax.resource.cci.Streamable;

public class GenericRecord implements Streamable, Record {
    private static final long serialVersionUID = -7600580218655773637L;
    private byte[] commarea = null;
    private String recordName;

    public GenericRecord() {
    }

    public GenericRecord(byte[] comm) {
        setCommarea(comm);
    }

    public void setCommarea(byte[] comm) {
        commarea = (byte[]) comm.clone();
    }

    public byte[] getCommarea() {
        return commarea;
    }

    public void read(InputStream in) throws IOException {
        commarea = null;

        int total = in.available();
        if (total > 0) {
            commarea = new byte[total];
            in.read(commarea);
        }
    }

    public void write(OutputStream out) throws IOException {
        if (commarea != null) {
```

```

        out.write(commarea);
        out.flush();
    }
}

@Override
public GenericRecord clone() throws CloneNotSupportedException {
    return new GenericRecord(getCommarea());
}

@Override
public String getRecordName() {
    return recordName;
}

@Override
public void setRecordName(String recordName) {
    this.recordName = recordName;
}

@Override
public void setRecordShortDescription(String arg0) {
}

@Override
public String getRecordShortDescription() {
    return null;
}
}

```

3.3. サブレット・プログラム

以下は、EJBを呼び出すサブレット・プログラムです。CtgXaのdoXATest()を呼び出して、その結果をWebブラウザー画面に表示します。

<ejb/client/HelloClient.java>

```

package ejb.client;

import java.io.IOException;
import java.io.PrintWriter;
import javax.ejb.EJB;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

```

```

import ejb.server.CtgXa;

public class HelloClient extends HttpServlet {
    private static final long serialVersionUID = 5385358571284268459L;
    @EJB(mappedName="ejb.server.CtgXa")
    private CtgXa ctgXaTest;

    protected void processRequest(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
        PrintWriter out = response.getWriter();
        try {
            String result = ctgXaTest.doXATest("TestData");

            response.setContentType("text/html");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>HelloClient</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h1>" + result + "</h1>");
            out.println("</body>");
            out.println("</html>");
            out.close();

        } catch(Exception ex){
            response.setContentType("text/plain");
            ex.printStackTrace(out);
        }
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response)

    throws ServletException, IOException {
        processRequest(request, response);
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)

    throws ServletException, IOException {
        processRequest(request, response);
    }
}

```

3.4. リソース・アダプター設定ファイル

以下は、リソース・アダプターの設定ファイルです。各config-property-name項目の詳細については、[「1.3. リソース・アダプターの構成」](#)のリソース・アダプターの設定項目を参照してください。

テスト環境に合わせて、host、host_port、regionName、txHelperClassName項目は必ず設定してください。

<ra.xml>

```
<?xml version="1.0" encoding="UTF-8"?>
<connector id="Connector_ID" version="1.5"
xmlns="http://java.sun.com/xml/ns/j2ee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
http://java.sun.com/xml/ns/j2ee/connector_1_5.xsd">
  <description>OFCTG Resource Adapter</description>
  <display-name>OFCTG_RA</display-name>
  <vendor-name>TmaxSoft</vendor-name>
  <eis-type>TP Monitor</eis-type>
  <resourceadapter-version>1.0.0.0</resourceadapter-version>
  <resourceadapter>
    <resourceadapter-class>
      com.tmax.connector.spi.TmaxResourceAdapterImpl
    </resourceadapter-class>
    <config-property>
      <config-property-name>port</config-property-name>
      <config-property-type>java.lang.Integer</config-property-type>
      <config-property-value>6557</config-property-value>
    </config-property>
    <config-property>
      <config-property-name>fdlfile</config-property-name>
      <config-property-type>java.lang.String</config-property-type>
      <config-property-value>/home/openframe/samples/tmax.fdl</config-property-value>
    </config-property>
    <outbound-resourceadapter>
      <connection-definition>
        <managedconnectionfactory-class>
          com.tmax.ofctg.connector.cics.ECIManagedConnectionFactory
        </managedconnectionfactory-class>
        <connectionfactory-interface>
          javax.resource.cci.ConnectionFactory
        </connectionfactory-interface>
        <connectionfactory-impl-class>
          com.tmax.ofctg.connector.cics.ECIConnectionFactory
        </connectionfactory-impl-class>
        <connection-interface>
```

```

    javax.resource.cci.Connection
</connection-interface>
<connection-impl-class>
    com.tmax.ofctg.connector.cics.ECIConnection
</connection-impl-class>
<config-property>
    <config-property-name>host</config-property-name>
    <config-property-type>java.lang.String</config-property-type>
    <config-property-value>192.168.100.200</config-property-value>
</config-property>
<config-property>
    <config-property-name>host_port</config-property-name>
    <config-property-type>java.lang.Integer</config-property-type>
    <config-property-value>9000</config-property-value>
</config-property>
<config-property>
    <config-property-name>backup</config-property-name>
    <config-property-type>java.lang.String</config-property-type>
    <config-property-value>192.168.100.201</config-property-value>
</config-property>
<config-property>
    <config-property-name>backup_port</config-property-name>
    <config-property-type>java.lang.Integer</config-property-type>
    <config-property-value>9000</config-property-value>
</config-property>
<config-property>
    <config-property-name>support_xa</config-property-name>
    <config-property-type>java.lang.Boolean</config-property-type>
    <config-property-value>true</config-property-value>
</config-property>
<config-property>
    <config-property-name>txHelperClassName</config-property-name>
    <config-property-type>java.lang.String</config-property-type>
    <config-property-value>jeus.transaction.TxHelper</config-property-value>
</config-property>
<config-property>
    <config-property-name>regionName</config-property-name>
    <config-property-type>java.lang.String</config-property-type>
    <config-property-value>OSC0IVP1</config-property-value>
</config-property>
<config-property>
    <config-property-name>headerType</config-property-name>
    <config-property-type>java.lang.String</config-property-type>
    <config-property-value>extendedV4</config-property-value>
</config-property>
</config-property>

```

```

        <config-property-name>connectTimeout</config-property-name>
        <config-property-type>java.lang.Integer</config-property-type>
        <config-property-value>30000</config-property-value>
    </config-property>
    <config-property>
        <config-property-name>fdlFile</config-property-name>
        <config-property-type>java.lang.String</config-property-type>
        <config-property-value>/home/openframe/samples/tmax.fdl</config-property-value>
    </config-property>
    <config-property>
        <config-property-name>xidMapperPath</config-property-name>
        <config-property-type>java.lang.String</config-property-type>
        <config-property-value>/home/openframe/xalog</config-property-value>
    </config-property>
    <config-property>
        <config-property-name>xidMapperFile</config-property-name>
        <config-property-type>java.lang.String</config-property-type>
        <config-property-value>webtxidmapper</config-property-value>
    </config-property>
    <config-property>
        <config-property-name>logLevel</config-property-name>
        <config-property-type>java.lang.String</config-property-type>
        <config-property-value>DEV</config-property-value>
    </config-property>
    <config-property>
        <config-property-name>logDir</config-property-name>
        <config-property-type>java.lang.String</config-property-type>
        <config-property-value>/home/openframe/log</config-property-value>
    </config-property>
    <config-property>
        <config-property-name>logFile</config-property-name>
        <config-property-type>java.lang.String</config-property-type>
        <config-property-value>webt.log</config-property-value>
    </config-property>
    <config-property>
        <config-property-name>logValidDays</config-property-name>
        <config-property-type>java.lang.Integer</config-property-type>
        <config-property-value>1</config-property-value>
    </config-property>
    <config-property>
        <config-property-name>connectionName</config-property-name>
        <config-property-type>java.lang.String</config-property-type>
        <config-property-value>tmax1</config-property-value>
    </config-property>
</connection-definition>
<transaction-support>XATransaction</transaction-support>

```

```
<reauthentication-support>false</reauthentication-support>
</outbound-resourceadapter>
<inbound-resourceadapter>
  <messageadapter>
    <messagelistener>
      <messagelistener-type>
        javax.resource.cci.MessageListener
      </messagelistener-type>
      <activation-spec>
        <activation-spec-class>
          com.tmax.connector.spi.TmaxActivationSpec
        </activation-spec-class>
        <required-config-property>
          <config-property-name>serviceName</config-property-name>
        </required-config-property>
      </activation-spec>
    </messagelistener>
  </messageadapter>
</inbound-resourceadapter>
</resourceadapter>
</connector>
```


索引

C

- CCILocalTransactionクラス, 8
 - メソッド, 8
- CICSConnectionFactoryクラス, 9
- CICSConnectionSpecクラス, 9
 - メソッド, 9
- CICSConnectionクラス, 9
- CICSInteractionSpecクラス, 11
 - メソッド, 12
- CICSInteractionクラス, 10
 - メソッド, 11
- CICSManagedConnectionFactoryクラス, 12
- CICSResourceExceptionクラス, 13
 - メソッド, 13

E

- ECIConnectionFactoryクラス, 15
 - メソッド, 15
- ECIConnectionSpecクラス, 15
 - コンストラクター, 16
 - メソッド, 16
- ECIConnectionクラス, 14
 - メソッド, 14
- ECIInteractionSpecクラス, 18
 - コンストラクター, 18
 - メソッド, 19
- ECIInteractionクラス, 16
 - メソッド, 16
- ECIManagedConnectionFactoryクラス, 21
- ECIResourceAdapterRcクラス, 21
 - 応答コード, 22
 - 定数, 23
 - メソッド, 22

O

- OpenFrame CTGのプロセス, 1

R

- ra.xml, 32

ら

- リソース・アダプターの構成, 2

